| | |
|---|---|
| **Name of Student:Husam Nour** | **PRN:25070122086** |
| **Semester: III** | **Year : AY 2026-27** |
| **Subject: Programming with Java** | |
| **ASSIGNMENT No : 27** | |
| **TITLE :** Develop a Java application demonstrating socket programming for communication between a client and a server | |

**Problem Statement**

Develop a Java application demonstrating socket programming for communication between a client and a server.

**Learning Objectives**

To understand socket programming in Java and implement communication between a client and a server using sockets.

**Theory**

Socket programming enables communication between computers over a network using TCP/IP protocols. Java provides the Socket class for client side communication and the ServerSocket class for server-side communication. A server listens for client requests on a specific port, while clients establish connections using the server's IP address and port number. Once connected, data is exchanged using input and output streams. Socket programming supports reliable communication between distributed systems and is widely used in chat applications, online gaming, file transfer systems, remote monitoring, and client-server architectures.

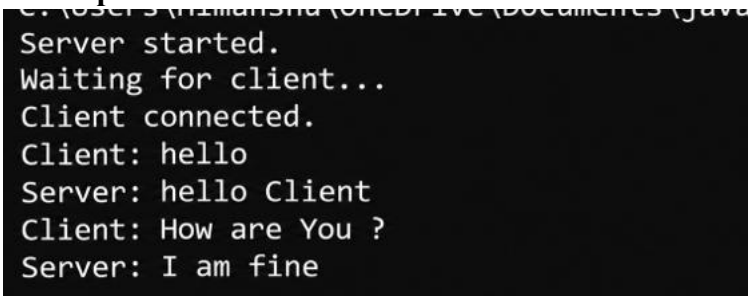
**1.Create a simple client-server chat applica on using sockets**.

**Programme:**

```
import java.io.*; import java.net.*; public class ChatServer { public static void main(String[] args)
{ try {
ServerSocket serverSocket = new ServerSocket(5050);
System.out.println("Server started.");
System.out.println("Waiting for client...");
Socket socket = serverSocket.accept();
System.out.println("Client connected.");
```

```
BufferedReader input = new BufferedReader( new
InputStreamReader(socket.getInputStream())
);
PrintWriter output = new
PrintWriter( socket.getOutputStream(), true
);
BufferedReader keyboard = new
BufferedReader( new
InputStreamReader(System.in)
);
String clientMessage; String serverMessage; while (true) { clientMessage = input.readLine();
if (clientMessage == null || clientMessage.equalsIgnoreCase("bye"))
{ break;
}
System.out.println("Client: " + clientMessage); System.out.print("Server: ");
serverMessage = keyboard.readLine(); output.println(serverMessage); if
(serverMessage.equalsIgnoreCase("bye"))
{ break;
}
}
socket.close(); serverSocket.close();
System.out.println("Chat ended.");
} catch (IOException e)
{ System.out.println("Error: " + e.getMessage());
}
```

**}**
**}**
**Output:**

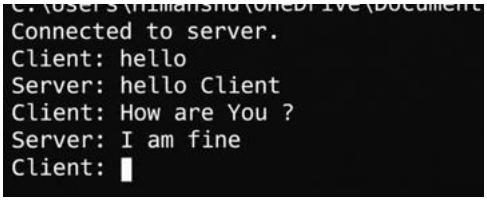



**Programme:**

**import java.io.*; import java.net.*; public class ChatClient { public static void main(String[] args)**
**{ try {**
**Socket socket = new Socket("localhost", 5050);**
**System.out.println("Connected to server.");**
**BufferedReader input = new BufferedReader( new InputStreamReader(socket.getInputStream())**
**);**
**PrintWriter output = new**
**PrintWriter( socket.getOutputStream(), true**
**);**
**BufferedReader keyboard = new**
**BufferedReader( new**
**InputStreamReader(System.in)**
**);**
**String clientMessage;**
**String serverMessage;**

```
while (true)
{ System.out.print("Client: "); clientMessage = keyboard.readLine();
output.println(clientMessage); if (clientMessage.equalsIgnoreCase("bye"))
{ break;
}
serverMessage = input.readLine(); System.out.println("Server: " + serverMessage); if
(serverMessage.equalsIgnoreCase("bye")) { break;
}
}
socket.close();
System.out.println("Chat ended.");
} catch (IOException e)
{ System.out.println("Error: " + e.getMessage());
}
}
}
```

**Output:**

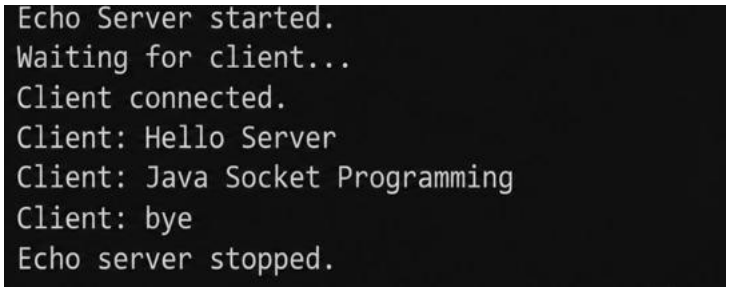



**2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.**

**Programme:**

```
import java.io.*; import java.net.*; public class EchoServer { public static void main(String[]
args)
{ try {
ServerSocket serverSocket = new ServerSocket(6000);
System.out.println("Echo Server started.");
System.out.println("Waiting for client...");
Socket socket = serverSocket.accept();
System.out.println("Client connected.");
BufferedReader input = new BufferedReader( new
InputStreamReader(socket.getInputStream()));
PrintWriter output = new PrintWriter( socket.getOutputStream(), true); String message;
while ((message = input.readLine()) != null) { System.out.println("Client: " + message);
output.println(message); if (message.equalsIgnoreCase("bye"))
{ break;
}
}
socket.close(); serverSocket.close();
System.out.println("Echo server stopped.");
} catch (IOException e)
{ System.out.println("Error: " + e.getMessage());
```

}
}
}
**Output:**

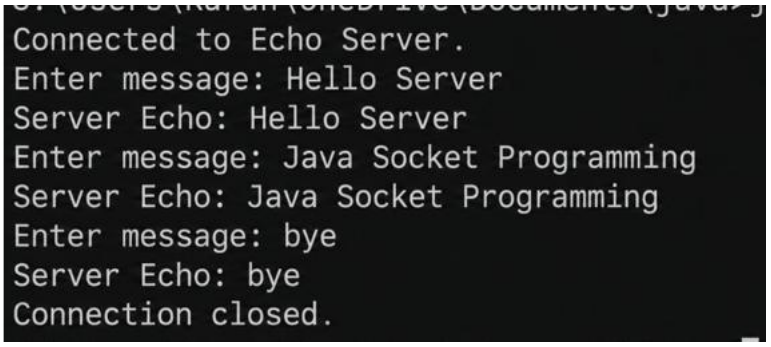



**Programme:**

**import java.io.*; import java.net.*; public class EchoClient { public static void main(String[] args)**
**{ try {**
**Socket socket = new Socket("localhost", 6000);**
**System.out.println("Connected to Echo Server.");**
**BufferedReader keyboard = new BufferedReader( new InputStreamReader(System.in));**
**BufferedReader input = new BufferedReader( new InputStreamReader(socket.getInputStream()));**
**PrintWriter output = new PrintWriter( socket.getOutputStream(), true); while (true) {**
**System.out.print("Enter message: ");**
**String message = keyboard.readLine();**

```
output.println(message);
String response = input.readLine();
System.out.println("Server Echo: " + response); if (message.equalsIgnoreCase("bye")) {
break;
}
}
socket.close();
System.out.println("Connection closed.");
} catch (IOException e)
{ System.out.println("Error: " + e.getMessage());
}
}
}
```

**Output:**




**Conclusion:**

The programs successfully demonstrate socket programming in Java. The first exercise establishes a two-way client-server chat using Socket and ServerSocket. The second exercise demonstrates an echo system, where the server receives a message from the client and sends the same message back.